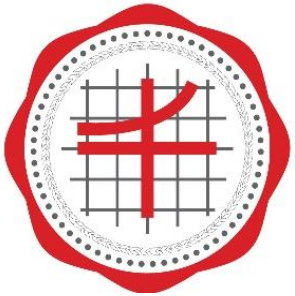
CPE 121 Mobile Application Developments

บทที่ 12

การเชื่อมต่อกับเหตุการณ์ (Event-Handling)



Faculty of Engineering
SRINAKHARINWIROT UNIVERSITY

Pramual Choorat, Ph.D.
Department of Computer Engineering

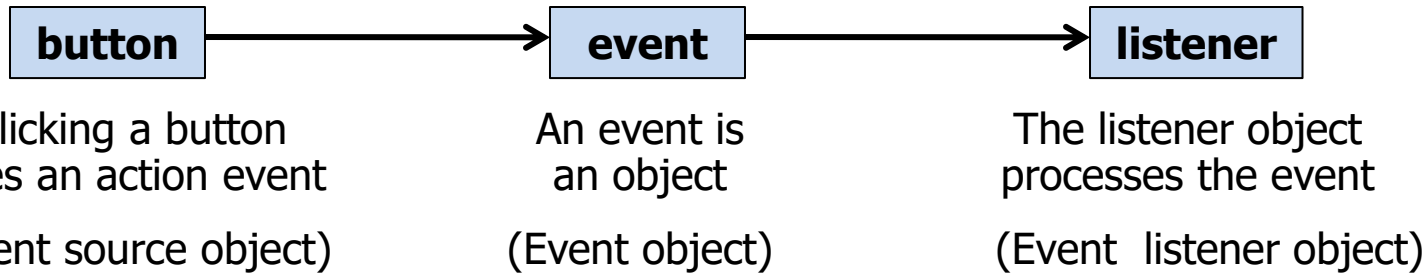
Event Handling

Event Handling เป็นการจัดการในส่วนโต้ตอบระหว่างผู้ใช้ กับ GUI ด้วย เช่น การคลิกที่ปุ่ม Button หรือการกด Enter บนแป้นพิมพ์ที่ Text Field เป็นต้น โดยจะต้องตรวจจับเหตุการณ์ก่อนว่าผู้ใช้มีการกระทำกับคอมโพเนนต์ที่เหตุการณ์ใดบ้าง ซึ่งเป็นหน้าที่ของออบเจ็กต์ที่เป็น Event Listener จากนั้นนำ Event Listener ไปผูกติดกับคอมโพเนนต์ เช่น ต้องการตรวจจับเหตุการณ์ที่ปุ่ม CloseButton โดยสร้างออบเจ็กต์จากคลาส ButtonListener เพื่อให้ออบเจ็กต์ดังกล่าวทำหน้าที่เป็น Event Listener และนำไปผูกติดกับปุ่ม CloseButton โดยมีรูปแบบดังนี้

buttonName.**addActionListener**(new ButtonListener());

โดยที่

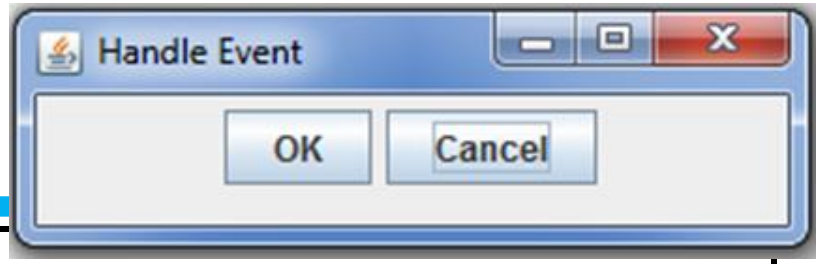
buttonName	เป็นชื่อออบเจ็กต์ที่ประกาศจากคลาส JButton
ButtonListener	เป็นออบเจ็กต์จากคลาส ButtonListener



ตัวอย่าง : Handle Event

```
import javax.swing.*;
import java.awt.event.*;
public class HandleEvent extends JFrame {
  public HandleEvent() {
    // Create two buttons
    JButton jbtOK = new JButton("OK"); JButton jbtCancel = new JButton("Cancel");

   // Create a panel to hold buttons
    JPanel panel = new JPanel();     panel.add(jbtOK);     panel.add(jbtCancel);
    add(panel);

    // Register listeners
    OKListenerClass listener1 = new OKListenerClass();
    CancelListenerClass listener2 = new CancelListenerClass();
    jbtOK.addActionListener(listener1);
    jbtCancel.addActionListener(listener2);
  }
```

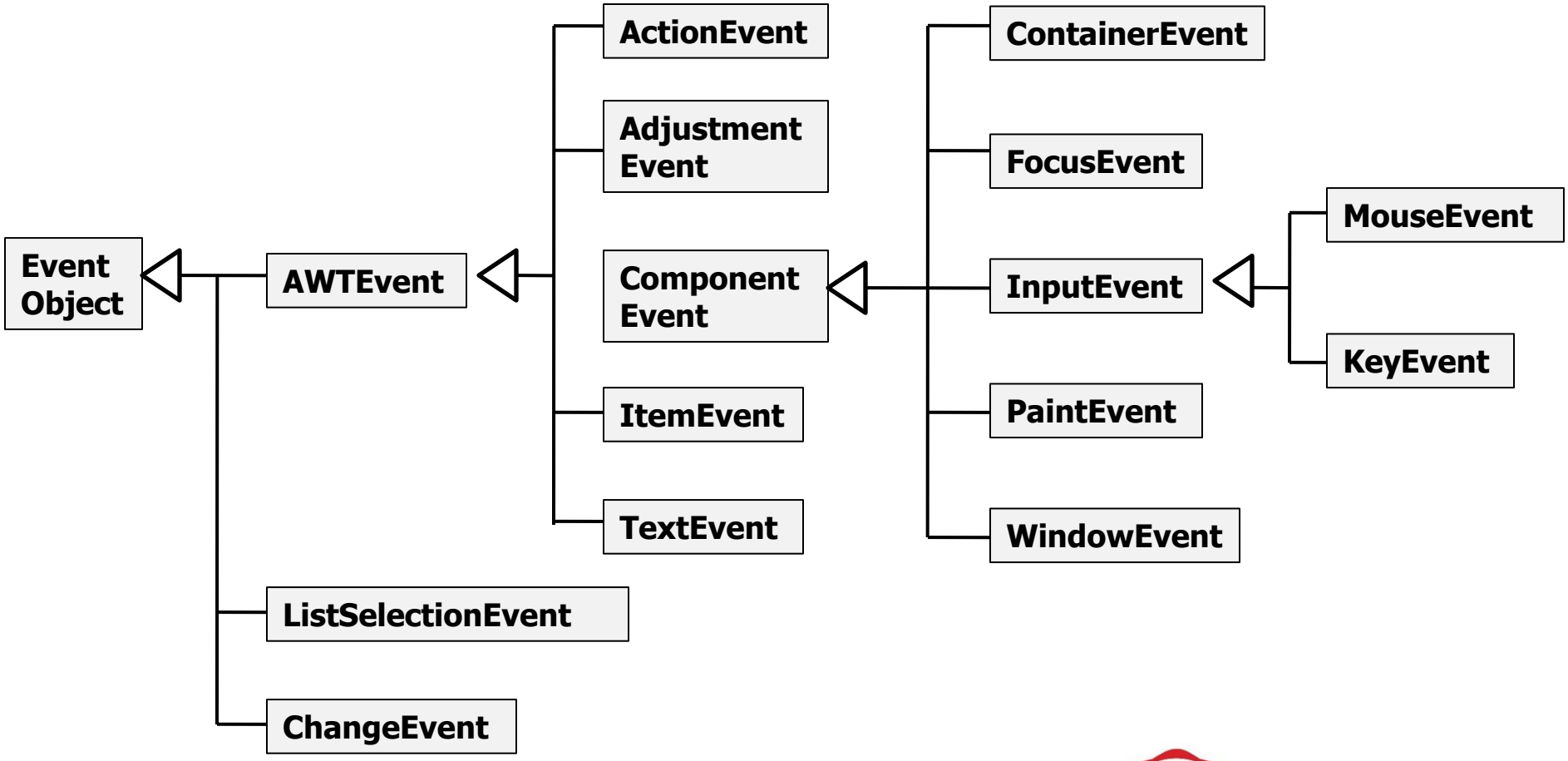
ตัวอย่าง : Handle Event




```
public static void main(String[] args) {
    JFrame frame = new HandleEvent();
    frame.setTitle("Handle Event");
    frame.setSize(200, 150);
    frame.setLocation(200, 100);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);   }   }

class OKListenerClass implements ActionListener {
  @Override
  public void actionPerformed(ActionEvent e) {
    System.out.println("OK button clicked");
  }}

class CancelListenerClass implements ActionListener {
  @Override
  public void actionPerformed(ActionEvent e) {
    System.out.println("Cancel button clicked");
  } }
```

```
> run HandleEvent
OK button clicked
Cancel button clicked
```

Events และ Event Sources

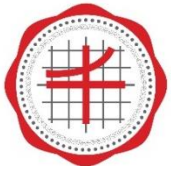


การเลือกใช้ Event Handlers

User Action	Source Object	Event Type Fired	Listener Interface	Listener Interface Methods
Click a button	JButton	ActionEvent	ActionListener	actionPerformed(ActionEvent e)
Press Enter in a text field	JTextField	ActionEvent	ActionListener	actionPerformed(ActionEvent e)
Select a new item	JComboBox	ActionEvent ItemEvent	ActionListener ItemListener	actionPerformed(ActionEvent e) itemStateChanged(ItemEvent e)
Check or uncheck	JCheckBox	ActionEvent ItemEvent	ActionListener ItemListener	actionPerformed(ActionEvent e) itemStateChanged(ItemEvent e)
Select a new item	JComboBox	ActionEvent ItemEvent	ActionListener ItemListener	actionPerformed(ActionEvent e) itemStateChanged(ItemEvent e)
Key pressed	Component	KeyEvent	KeyListener	keyPressed(KeyEvent e)
Key released				keyReleased(KeyEvent e)
Key typed				keyTyped(KeyEvent e)

การเลือกใช้ Event Handlers

User Action	Source Object	Event Type Fired	Listener Interface	Listener Interface Methods
Mouse pressed	Component	MouseEvent	MouseListener	mousePressed(MouseEvent e)
Mouse released				mouseReleased(MouseEvent e)
Mouse clicked				mouseClicked(MouseEvent e)
Mouse entered				mouseEntered(MouseEvent e)
Mouse exited				mouseExited(MouseEvent e)
Mouse moved			MouseMotionListener	mouseMoved(MouseEvent e)
Mouse dragged				mouseDragged(MouseEvent e)

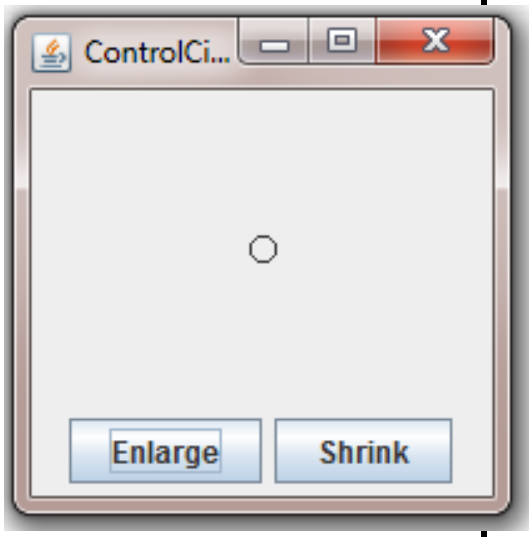


ตัวอย่าง : Controlling Balls 1

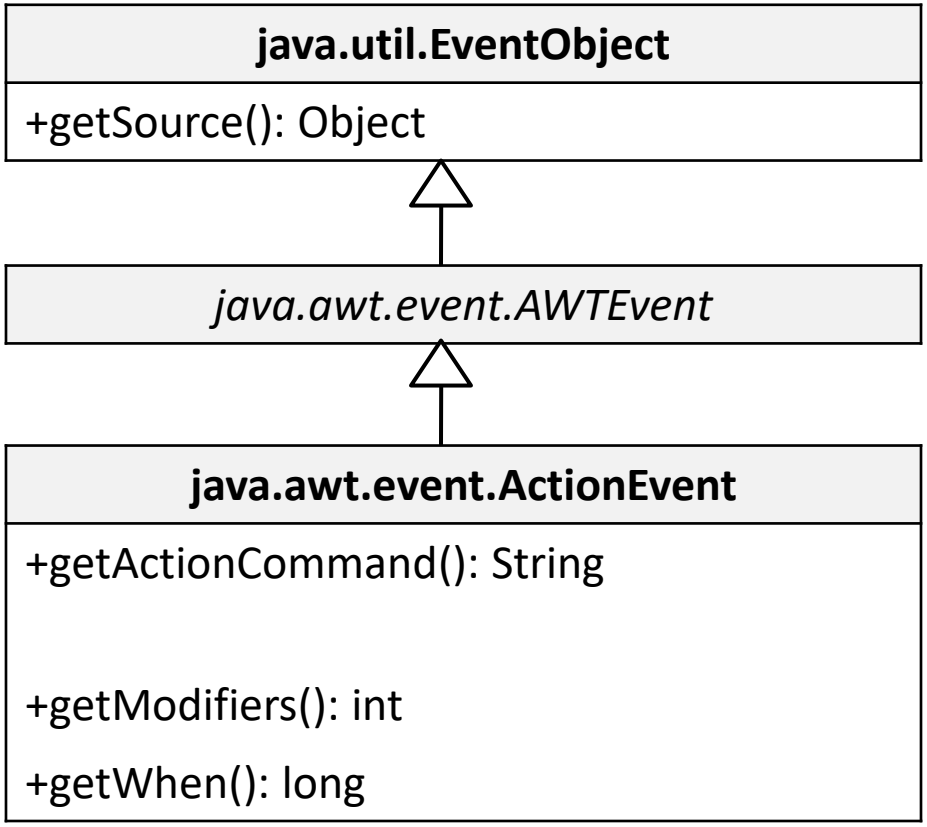
```
import javax.swing.*;
import java.awt.*;
public class ControlCircleWithoutEventHandling extends JFrame {
  private JButton jbtEnlarge = new JButton("Enlarge");
  private JButton jbtShrink = new JButton("Shrink");
  private CirclePanel canvas = new CirclePanel();

  public ControlCircleWithoutEventHandling() {
    JPanel panel = new JPanel(); // Use the panel to group buttons
    panel.add(jbtEnlarge);
    panel.add(jbtShrink);

    this.add(canvas, BorderLayout.CENTER); // Add canvas to center
    this.add(panel, BorderLayout.SOUTH); // Add buttons to the frame
  }
```


ตัวอย่าง : Controlling Balls 1

```
 /** Main method */
  public static void main(String[] args) {
    JFrame frame = new ControlCircleWithoutEventHandling();
    frame.setTitle("ControlCircleWithoutEventHandling");
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(200, 200);
    frame.setVisible(true);   }
}
class CirclePanel extends JPanel {
  private int radius = 5; // Default circle radius
  @Override
  protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    g.drawOval(getWidth() / 2 - radius, getHeight() / 2 – radius, 2 * radius, 2 *
radius);
  }
}
```



java.awt.event.ActionEvent



- Returns the source object for the event.

- Returns the command string associated with this action. For a button, its text is the command string.

- Returns the modifier keys held down during this action event.

- Returns the timestamp when this event occurred. The time is the number of milliseconds since January 1, 1970, 00:00:00 GMT.

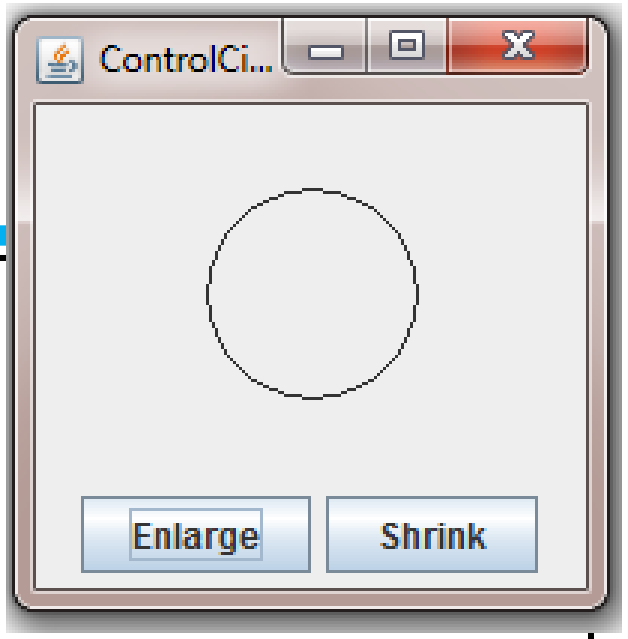
ตัวอย่าง : Controlling Balls 2

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class ControlCircle extends JFrame {
  private JButton jbtEnlarge = new JButton("Enlarge");
  private JButton jbtShrink = new JButton("Shrink");
  private CirclePanel canvas = new CirclePanel();

  public ControlCircle() {
    JPanel panel = new JPanel();   // Use the panel to group buttons
    panel.add(jbtEnlarge);
    panel.add(jbtShrink);
    this.add(canvas, BorderLayout.CENTER); // Add canvas to center
    this.add(panel, BorderLayout.SOUTH); // Add buttons to the frame

    jbtEnlarge.addActionListener(new EnlargeListener());
  }
```

ตัวอย่าง : Controlling Balls 2

```
/** Main method */
  public static void main(String[] args) {
    JFrame frame = new ControlCircle();
    frame.setTitle("ControlCircle");
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(200, 200);
    frame.setVisible(true);
  }

  class EnlargeListener implements ActionListener { // Inner class
     @Override
     public void actionPerformed(ActionEvent e) {
       canvas.enlarge();
     }
  }
```

ตัวอย่าง : Controlling Balls 2




```
class CirclePanel extends JPanel { // Inner class
    private int radius = 5; // Default circle radius

    /** Enlarge the circle */
    public void enlarge() {
      radius++;
      repaint();
    }
    @Override
    protected void paintComponent(Graphics g) {
      super.paintComponent(g);
      g.drawOval(getWidth() / 2 - radius, getHeight() / 2 – radius, 2 * radius, 2 *
radius);
    }
  }
}
```

Inner Class

การเขียนโปรแกรมในส่วนจัดการเหตุการณ์ สามารถจัดการให้อยู่ภายใน Inner Class โดยสร้างเป็นคลาสที่มีการ **implements** อินเตอร์เฟสซึ่งสอดคล้องกับเหตุการณ์ที่เกิดขึ้น และกำหนดหน้าที่การทำงานให้กับเมธอดในอินเตอร์เฟสนั้นๆ โดยมีรูปแบบดังนี้

```
private class ButtonListener implements ActionListener  {
    public void actionPerformed(ActionEvent e)  {
        if(e.getSource()==buttonName)  {
        }
    }
}
```

โดยที่

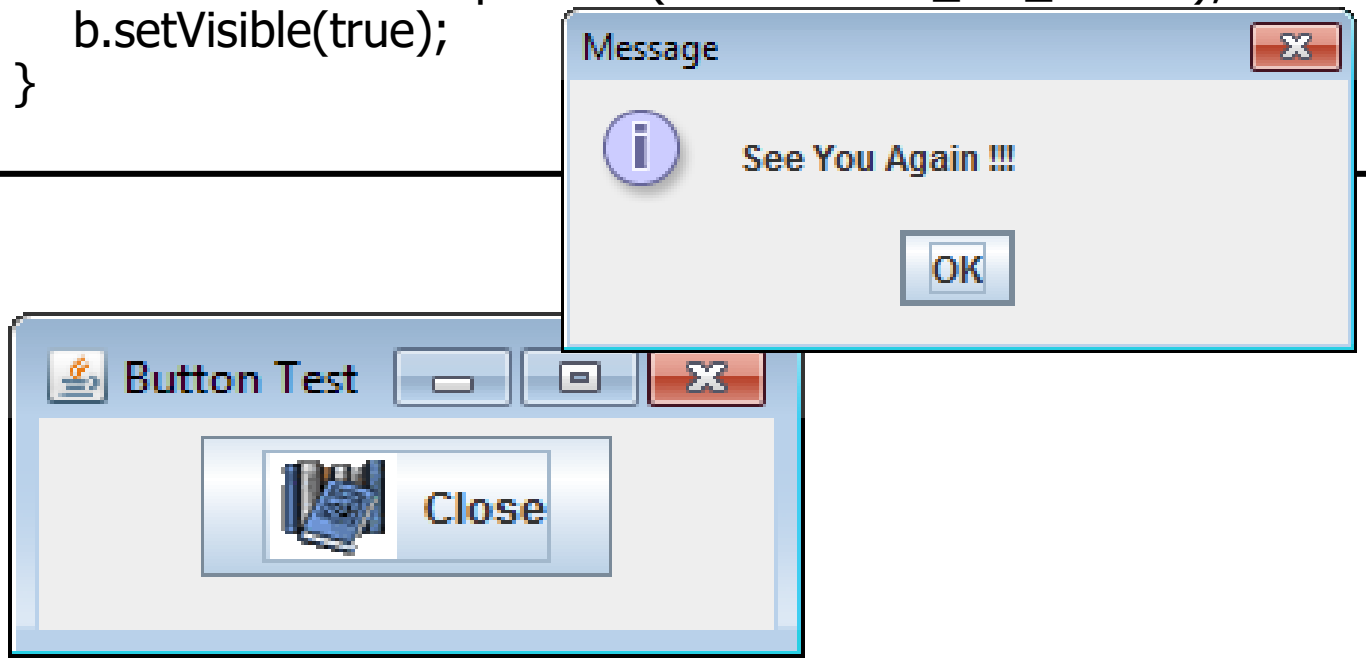
ButtonListener	เป็นออบเจ็กต์จากคลาส ButtonListener
ActionListener	เป็นเหตุการณ์ที่ต้องการตรวจจับ
actionPerformed	เป็นชื่อเมธอด
buttonName	เป็นชื่อออบเจ็กต์ที่ประกาศจากคลาส JButton

โปรแกรมตรวจจับการทำงานที่ปุ่ม Close

```
import java.awt.event.*;    import javax.swing.*;
public class CloseButtonTest extends JFrame {
    private JPanel p;  Icon ani;  JButton b;
    public CloseButtonTest(String title) {
        super(title);
        p = new JPanel();
        ani = new ImageIcon("Course.gif");
        b = new JButton("Close", ani);
        b.addActionListener(new ButtonListener());
        p.add(b);          add(p);
    }

 private class ButtonListener implements ActionListener {
        public void actionPerformed(ActionEvent e) {
            if (e.getSource() == b) {
                JOptionPane.showMessageDialog(null, "See You Again !!!");
                System.exit(0);
            }
        }
     }
```

โปรแกรมตรวจจับการทำงานที่ปุ่ม Close

```
public static void main(String args[]) {
        CloseButtonTest b = new CloseButtonTest("Button Test");
        b.setSize(170, 90);
        b.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        b.setVisible(true);
    }
}
```

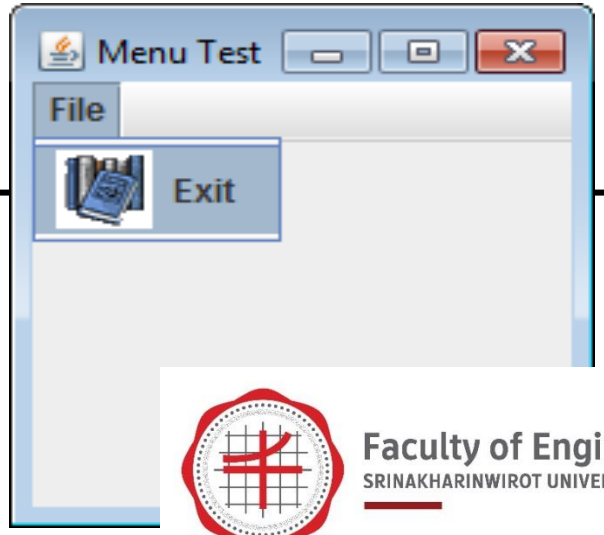


โปรแกรมตรวจจับการทำงานที่เมนู Exit

```
import javax.swing.*;
import java.awt.event.*;
public class ExitMenuTest extends JFrame {
    JMenuBar menubar;   JMenu menuFile;
    JMenuItem menuExit;

    public ExitMenuTest(String title) {
        super(title);
        menubar = new JMenuBar();
        menuFile = new JMenu("File");
        menuExit = new JMenuItem("Exit", new ImageIcon("Course.gif"));
        menuFile.add(menuExit);
        menubar.add(menuFile);
        setJMenuBar(menubar);
        menuExit.addActionListener(new MenuListener());
    }
```

โปรแกรมตรวจจับการทำงานที่เมนู Exit

```
    private class MenuListener implements ActionListener {
        public void actionPerformed(ActionEvent e) {
            Object source = e.getSource();
            if (source == menuExit) {
                System.exit(0);
            }
        }
    }
    public static void main(String args[]) {
        ExitMenuTest f = new ExitMenuTest("Menu Test");
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setSize(200, 200);
        f.setVisible(true);
    }
}
```



Anonymous Class Listeners

คลาส anonymous inner เป็น inner class ที่จะไม่มีการเขียนชื่อ โดยจะเป็นการรวมชื่อและทำการสร้างคลาสไว้ในอันเดียวกัน

```
public ControlCircle() {
  // Omitted
jbtEnlarge.addActionListener(
    new EnlargeListener());
}
class EnlargeListener
    implements ActionListener {
  @Override
  public void actionPerformed(ActionEvent e) {
    canvas.enlarge();
  }
}
```

```
public ControlCircle() {
  // Omitted
jbtEnlarge.addActionListener(
    new class EnlargeListener
    implements ActionListener () {
  @Override
  public void actionPerformed(ActionEvent e) {
    canvas.enlarge();
  }
  } );
}
```

ตัวอย่าง : AnonymousListener

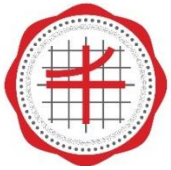
```
import javax.swing.*;
import java.awt.event.*;
public class AnonymousListenerDemo extends JFrame {
  public AnonymousListenerDemo() {
    // Create four buttons
    JButton jbtNew = new JButton("New");
    JButton jbtOpen = new JButton("Open");
    JButton jbtSave = new JButton("Save");
    JButton jbtPrint = new JButton("Print");

    // Create a panel to hold buttons
    JPanel panel = new JPanel();
    panel.add(jbtNew);
    panel.add(jbtOpen);
    panel.add(jbtSave);
    panel.add(jbtPrint);

    add(panel);
```

ตัวอย่าง : AnonymousListener

```
// Create and register anonymous inner class listener
    jbtNew.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
          System.out.println("Process New");
        }
      }
    );
    jbtOpen.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
          System.out.println("Process Open");
        }
      }
    );
```

ตัวอย่าง : AnonymousListener

```
    jbtSave.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
          System.out.println("Process Save");
        }
      }
    );

    jbtPrint.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
          System.out.println("Process Print");
        }
      }
    );
  }
```

ตัวอย่าง : AnonymousListener

```
  /** Main method */
  public static void main(String[] args) {
    JFrame frame = new AnonymousListenerDemo();
    frame.setTitle("AnonymousListenerDemo");
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.pack();
    frame.setVisible(true);
  }
}
```


```
> run AnonymousListenerDemo
Process New
Process Open
Process Save
Process Print
```

ตัวอย่าง : LoanCalculator

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.TitledBorder;

public class LoanCalculator extends JFrame {
  // Create text fields for interest rate, years
  // loan amount, monthly payment, and total payment
  private JTextField jtfAnnualInterestRate = new JTextField();
  private JTextField jtfNumberOfYears = new JTextField();
  private JTextField jtfLoanAmount = new JTextField();
  private JTextField jtfMonthlyPayment = new JTextField();
  private JTextField jtfTotalPayment = new JTextField();

  // Create a Compute Payment button
  private JButton jbtComputeLoan = new JButton("Compute Payment");
```

ตัวอย่าง : LoanCalculator

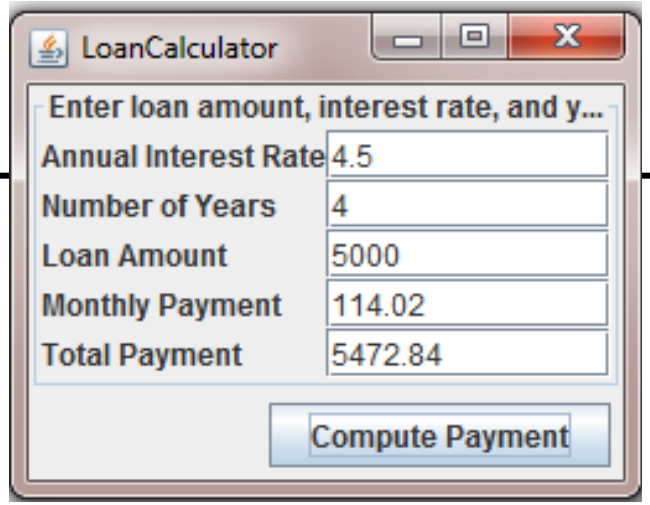
```
public LoanCalculator() {
  // Panel p1 to hold labels and text fields
  JPanel p1 = new JPanel(new GridLayout(5, 2));
  p1.add(new JLabel("Annual Interest Rate"));
  p1.add(jtfAnnualInterestRate);
  p1.add(new JLabel("Number of Years"));
  p1.add(jtfNumberOfYears);
  p1.add(new JLabel("Loan Amount"));
  p1.add(jtfLoanAmount);
  p1.add(new JLabel("Monthly Payment"));
  p1.add(jtfMonthlyPayment);
  p1.add(new JLabel("Total Payment"));
  p1.add(jtfTotalPayment);
  p1.setBorder(new TitledBorder("Enter loan amount, interest rate, and years"));

  // Panel p2 to hold the button
  JPanel p2 = new JPanel(new FlowLayout(FlowLayout.RIGHT));
  p2.add(jbtComputeLoan);
```

ตัวอย่าง : LoanCalculator

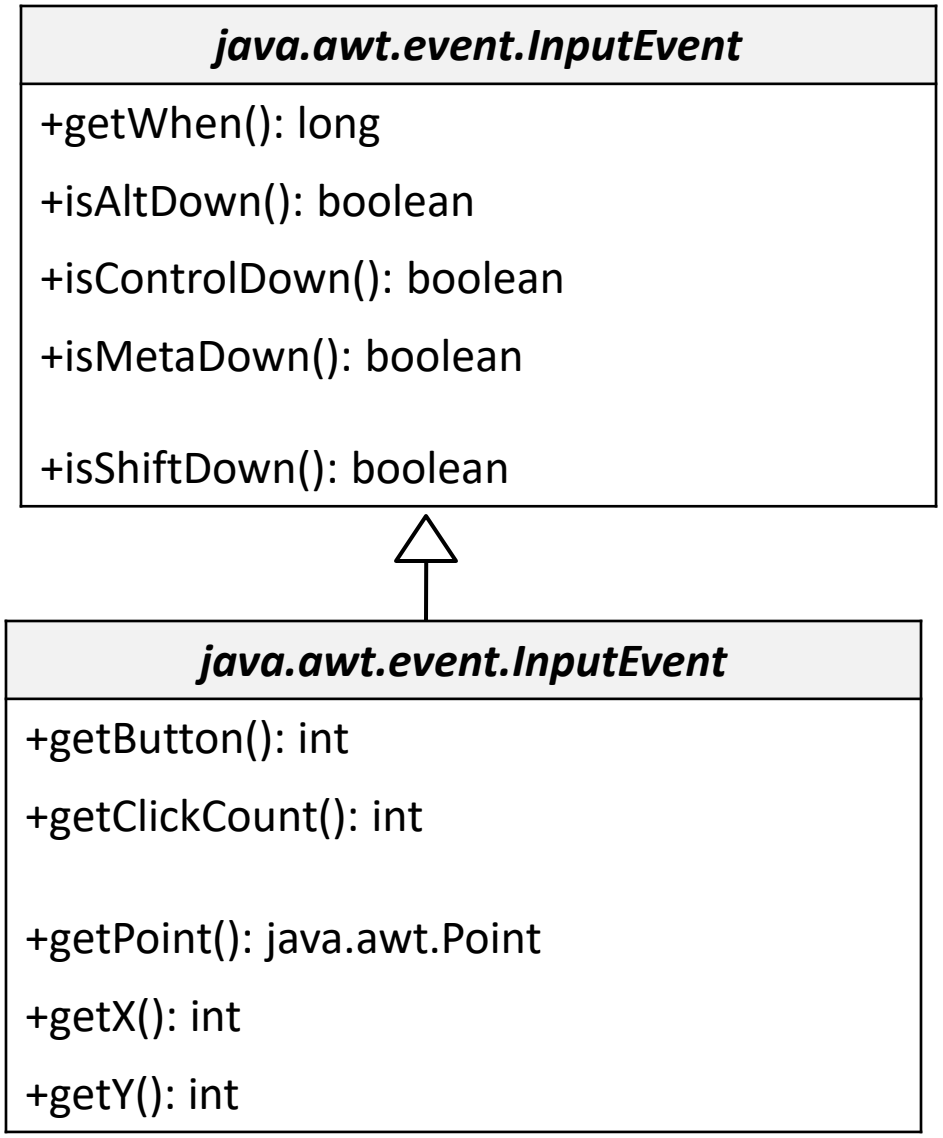
```
  // Add the panels to the frame
   add(p1, BorderLayout.CENTER);
   add(p2, BorderLayout.SOUTH);
   // Register listener
   jbtComputeLoan.addActionListener(new ButtonListener());
}
/** Handle the Compute Payment button */
private class ButtonListener implements ActionListener {
   @Override
   public void actionPerformed(ActionEvent e) {
     // Get values from text fields
     double interest =  Double.parseDouble(jtfAnnualInterestRate.getText());
     int year = Integer.parseInt(jtfNumberOfYears.getText());
     double loanAmount =  Double.parseDouble(jtfLoanAmount.getText());

     // Create a loan object
     Loan loan = new Loan(interest, year, loanAmount);
```

ตัวอย่าง : LoanCalculator

```
      // Display monthly payment and total payment
      jtfMonthlyPayment.setText(String.format("%.2f", loan.getMonthlyPayment()));
      jtfTotalPayment.setText(String.format("%.2f", loan.getTotalPayment()));
    }
  }
  public static void main(String[] args) {
  LoanCalculator frame = new LoanCalculator();
  frame.pack();
  frame.setTitle("LoanCalculator");
  frame.setLocationRelativeTo(null); // Center the frame
  frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  frame.setVisible(true);
  }
}
```



Mouse Events



- Returns the timestamp when this event occurred
- Returns true if the Alt key is pressed on this event.
- Returns true if the Control key is pressed on this event.
- Returns true if the Meta mouse button is pressed on this event.
- Returns true if the Shift key is pressed on this event.

- Indicates which mouse button has been clicked.
- Returns the number of mouse clicks associated with this event.
- Returns a Point object containing the *x- and y-coordinates*
- Returns the *x-coordinate of the* ***mouse point.***
- Returns the *y-coordinate of the* ***mouse point****.*

การเชื่อมต่อกับ Mouse Events

จาวาจะใช้ MouseListener และ MouseMotionListener สำหรับใช้ในการที่เชื่อมต่อกับ mouse events โดยที่

- MouseListener จะใช้เชื่อมต่อกับเหตุการณ์กรณีที่เมื่อเมาท์ถูกทำการกดปล่อย Enter Exit หรือคลิ๊กที่ตัวคอมโพเนนท์
- MouseMotionListener จะใช้เชื่อมต่อกับเหตุการณ์กรณีที่ทำการลากหรือเลื่อนเมาท์

การเชื่อมต่อกับ Mouse Events

<<interface>> *java.awt.event.MouseListener*	
+mousePressed(e: MouseEvent): void	- Invoked after the mouse button has been pressed on the source component.
+mouseReleased(e: MouseEvent): void	- Invoked after the mouse button has been released on the source component.
+mouseClicked(e: MouseEvent): void	- Invoked after the mouse button has been clicked (pressed and released) on the source component.
+mouseEntered(e: MouseEvent): void	- Invoked after the mouse enters the source component.
+mouseExited(e: MouseEvent): void	- Invoked after the mouse exits the source component.

<<interface>> *java.awt.event.MouseMotionListener*	
+mouseDragged(e: MouseEvent): void	- Invoked after a mouse button is moved with a button pressed.
+mouseMoved(e: MouseEvent): void	- Invoked after a mouse button is moved without a button pressed.

Example: MoveMessage

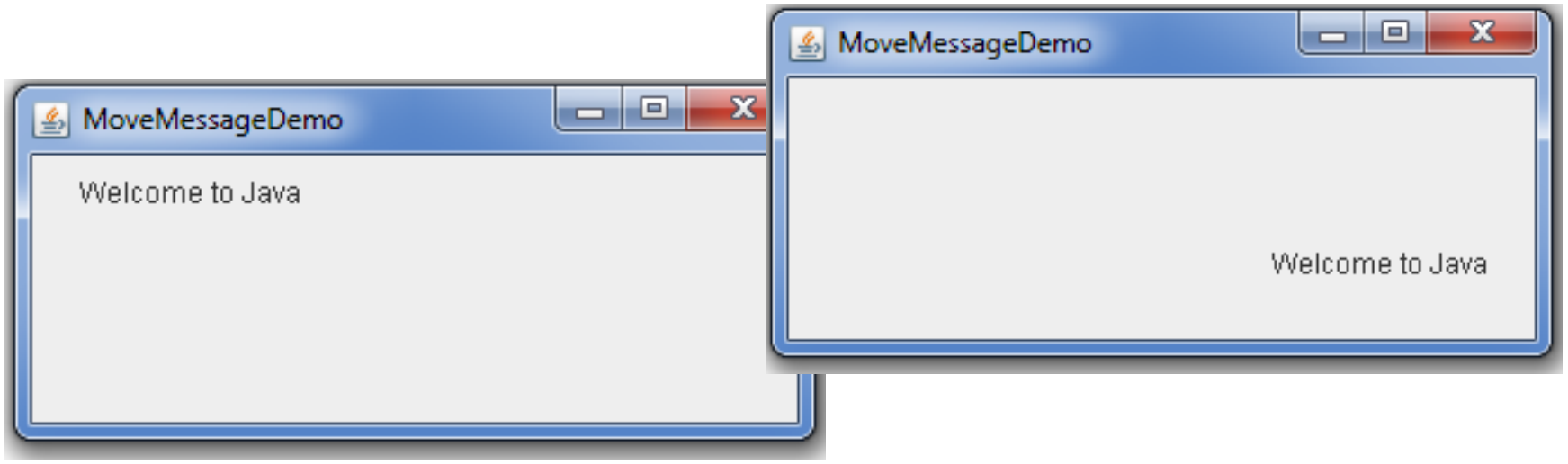
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class MoveMessageDemo extends JFrame {
  public MoveMessageDemo() {
    // Create a MovableMessagePanel instance for moving a message
    MovableMessagePanel p = new MovableMessagePanel ("Welcome to Java");
    // Place the message panel in the frame
    add(p);
  }
  /** Main method */
  public static void main(String[] args) {
    MoveMessageDemo frame = new MoveMessageDemo();
    frame.setTitle("MoveMessageDemo");
    frame.setSize(200, 100);
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
  }
```

Example: MoveMessage

```
// Inner class: MovableMessagePanel draws a message
static class MovableMessagePanel extends JPanel {
  private String message = "Welcome to Java";
  private int x = 20;    private int y = 20;

  /** Construct a panel to draw string s */
  public MovableMessagePanel(String s) {
    message = s;
    addMouseMotionListener(new MouseMotionListener() {
      @Override /** Handle mouse-dragged event */
      public void mouseDragged(MouseEvent e) {
        // Get the new location and repaint the screen
        x = e.getX();          y = e.getY();          repaint();
      }
      @Override /** Handle mouse-moved event */
      public void mouseMoved(MouseEvent e) {
      }
    });
  }
```

Example: MoveMessage

```
    @Override
    protected void paintComponent(Graphics g) {
      super.paintComponent(g);
      g.drawString(message, x, y);
    }
  }
}
```

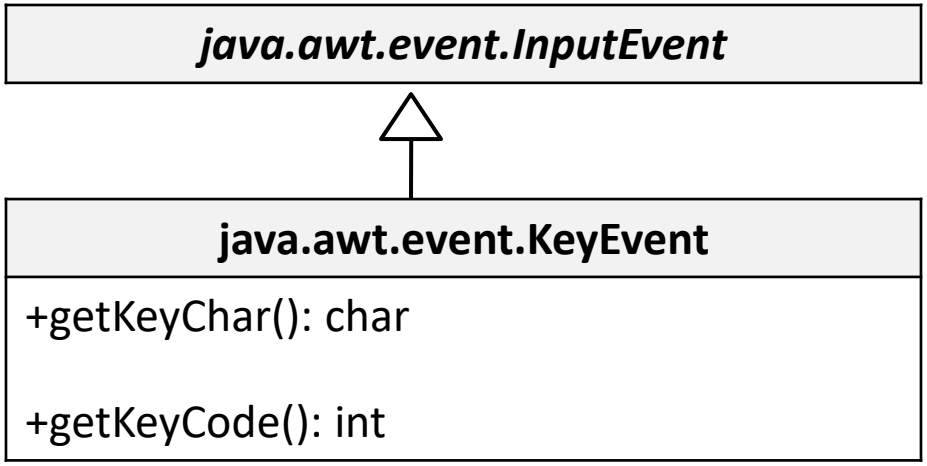


การเชื่อมต่อกับ KeyEvents

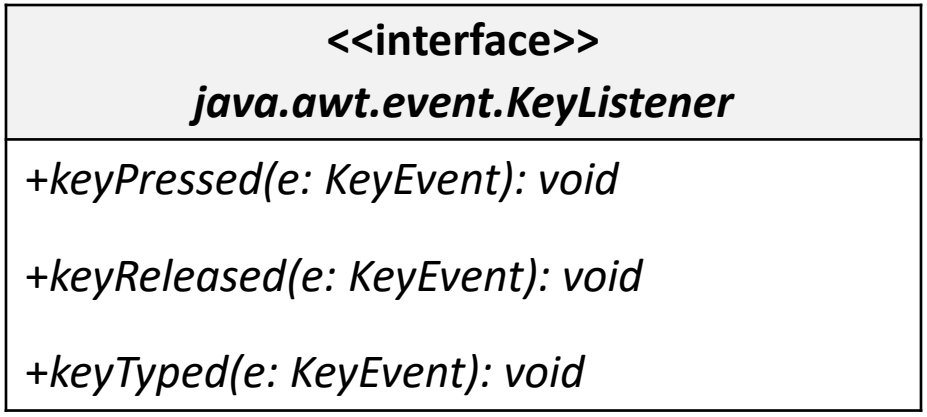
การเชื่อมต่อเหตุการณ์เข้ากับคีย์บอร์ด เราจะใช้วิธีการที่อยู่ในคลาส KeyListener ดังนี้

- keyPressed(KeyEvent e) จะถูกเรียกใช้เมื่อคีย์บอร์ดถูกทำการกด
- keyReleased(KeyEvent e)จะถูกเรียกใช้เมื่อคีย์บอร์ดถูกทำการปล่อย
- keyTyped(KeyEvent e)จะถูกเรียกใช้เมื่อคีย์บอร์ดโดนกดแล้วจากนั้นทำการปล่อย

คลาส KeyEvents และ KeyListener



- Returns the character associated with the key in this event.
- Returns the integer key code associated with the key in this event.



- Invoked after a key is pressed on the source component.
- Invoked after a key is released on the source component.
- Invoked after a key is pressed and then released on the source component.

Key Constants

Constant	Description	Constant	Description
VK_HOME	The Home key	**VK_SHIFT**	The Shift key
VK_END	The End key	**VK_BACK_SPACE**	The Backspace key
VK_PGUP	The Page Up key	**VK_CAPS_LOCK**	The Caps Lock key
VK_PGDN	The Page Down key	**VK_NUM_LOCK**	The Num Lock key
VK_UP	The up-arrow key	**VK_ENTER**	The Enter key
VK_DOWN	The down-arrow key	**VK_UNDEFINED**	The **keyCode** unknown
VK_LEFT	The left-arrow key	**VK_F1** to **VK_F12**	The function keys from F1 to F12
VK_RIGHT	The right-arrow key		
VK_ESCAPE	The Esc key	**VK_0** to **VK_9**	The number keys from 0 to 9
VK_TAB	The Tab key	**VK_A** to **VK_Z**	The letter keys from A to Z
VK_CONTROL	The Control key		

ตัวอย่าง : KeyEvent

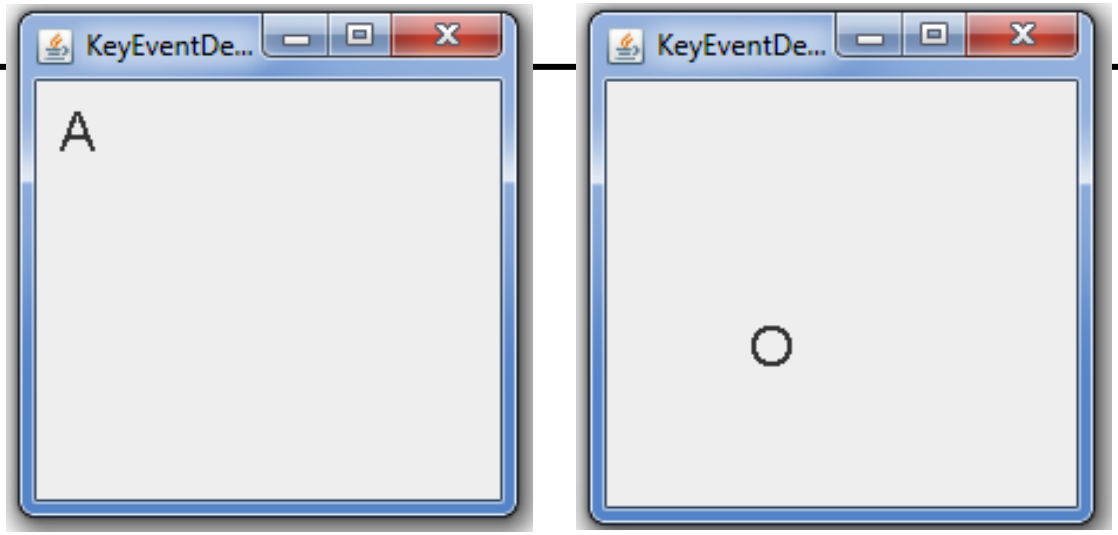
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class KeyEventDemo extends JFrame {
  private KeyboardPanel keyboardPanel = new KeyboardPanel();
  /** Initialize UI */
  public KeyEventDemo() {
    // Add the keyboard panel to accept and display user input
    add(keyboardPanel);
    // Set focus
    keyboardPanel.setFocusable(true);
  }  /** Main method */
  public static void main(String[] args) {
    KeyEventDemo frame = new KeyEventDemo();
    frame.setTitle("KeyEventDemo");
    frame.setSize(300, 300);
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);   }
```

ตัวอย่าง : KeyEvent

```
// Inner class: KeyboardPanel for receiving key input
  static class KeyboardPanel extends JPanel {
    private int x = 100;    private int y = 100;
    private char keyChar = 'A'; // Default key
    public KeyboardPanel() {
      addKeyListener(new KeyAdapter() {
        @Override
        public void keyPressed(KeyEvent e) {
          switch (e.getKeyCode()) {
            case KeyEvent.VK_DOWN: y += 10; break;
            case KeyEvent.VK_UP: y -= 10; break;
            case KeyEvent.VK_LEFT: x -= 10; break;
            case KeyEvent.VK_RIGHT: x += 10; break;
            default: keyChar = e.getKeyChar();
          }
          repaint();
        }
      });
    }
```

ตัวอย่าง : KeyEvent

```
    @Override
    protected void paintComponent(Graphics g) {
      super.paintComponent(g);

      g.setFont(new Font("TimesRoman", Font.PLAIN, 24));
      g.drawString(String.valueOf(keyChar), x, y);
    }
  }
}
```



ตัวอย่าง : ControlCircleWithMouseAndKey

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class ControlCircleWithMouseAndKey extends JFrame {
  private JButton jbtEnlarge = new JButton("Enlarge");
  private JButton jbtShrink = new JButton("Shrink");
  private CirclePanel canvas = new CirclePanel();
  public ControlCircleWithMouseAndKey() {
    JPanel panel = new JPanel(); // Use the panel to group buttons
    panel.add(jbtEnlarge);     panel.add(jbtShrink);
    this.add(canvas, BorderLayout.CENTER); // Add canvas to center
    this.add(panel, BorderLayout.SOUTH); // Add buttons to the frame
    jbtEnlarge.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent e) {
        canvas.enlarge();
        canvas.requestFocusInWindow();
      }
    });
```

ตัวอย่าง : ControlCircleWithMouseAndKey

```
jbtShrink.addActionListener(new ActionListener() {
   @Override
   public void actionPerformed(ActionEvent e) {
     canvas.shrink();
     canvas.requestFocusInWindow();
   }
});

canvas.addMouseListener(new MouseAdapter() {
  @Override
   public void mouseClicked(MouseEvent e) {
     if (e.getButton() == MouseEvent.BUTTON1)
       canvas.enlarge();
     else if (e.getButton() == MouseEvent.BUTTON3)
       canvas.shrink();
   }
});
```

ตัวอย่าง : ControlCircleWithMouseAndKey

```
    canvas.setFocusable(true);
    canvas.addKeyListener(new KeyAdapter() {
      @Override
      public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_UP)
          canvas.enlarge();
        else if (e.getKeyCode() == KeyEvent.VK_DOWN)
          canvas.shrink();
      }
    });
  }
  /** Main method */
  public static void main(String[] args) {
    JFrame frame = new ControlCircleWithMouseAndKey();
    frame.setTitle("ControlCircle");
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(200, 200);
    frame.setVisible(true);   }
```

ตัวอย่าง : ControlCircleWithMouseAndKey

```
class CirclePanel extends JPanel { // Inner class
  private int radius = 5; // Default circle radius
  /** Enlarge the circle */
  public void enlarge() {
    radius++;      repaint();
  }
  /** Shrink the circle */
  public void shrink() {
    if (radius >= 1) radius--;
    repaint();
  }
  @Override
  protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    g.drawOval(getWidth() / 2 - radius, getHeight() / 2 – radius,2 * radius, 2 *
radius);
  }
 }
}
```

คลาส Timer

คลาส Timer เป็นคลาสที่เป็นการนำเวลามาตรฐานที่อยู่ในระบบของเครื่องคอมพิวเตอร์มาใช้เพื่อช่วยในการสร้างโปรแกรมที่มีลักษณะของการเคลื่อนไหวหรือโปรแกรมที่จำเป็นต้องใช้ข้อมูลของเวลา โดยเมธอดในคลาสจะมีด้วยกันดังนี้

javax.swing.Timer	
+Timer(delay: int, listener: ActionListener)	- Creates a Timer object with a specified delay in milliseconds and an **ActionListener**.
+addActionListener(listener: ActionListener): void	- Adds an **ActionListener** to the timer.
+start(): void	- Starts this timer.
+stop(): void	- Stops this timer.
+setDelay(delay: int): void	- Sets a new delay value for this timer.

ตัวอย่าง : Animation

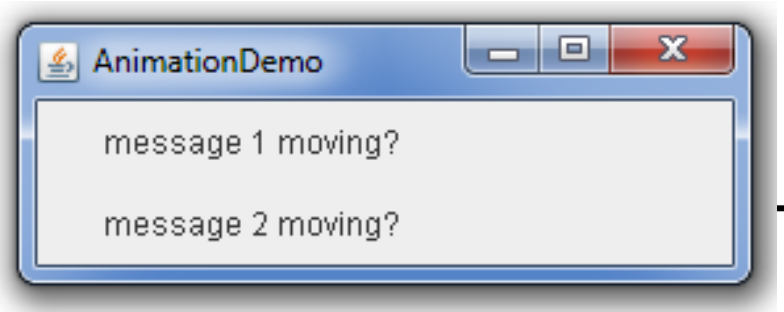
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class AnimationDemo extends JFrame {
  public AnimationDemo() {
    // Create two MovingMessagePanel for displaying two moving messages
    this.setLayout(new GridLayout(2, 1));
    add(new MovingMessagePanel("message 1 moving?"));
    add(new MovingMessagePanel("message 2 moving?"));
  }
  /** Main method */
  public static void main(String[] args) {
    AnimationDemo frame = new AnimationDemo();
    frame.setTitle("AnimationDemo");
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(280, 100);
    frame.setVisible(true);
  }
```

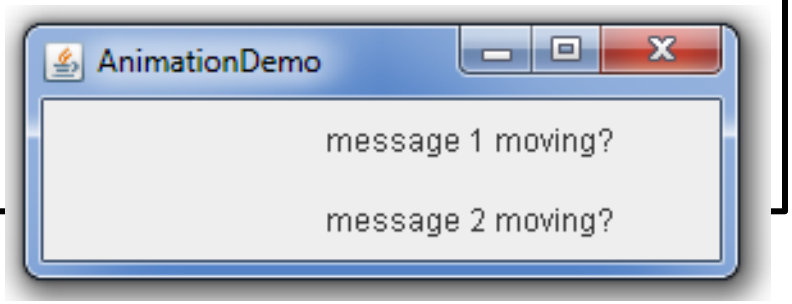
ตัวอย่าง : Animation

```
  // Inner class: Displaying a moving message
  static class MovingMessagePanel extends JPanel {
    private String message = "Welcome to Java";
    private int xCoordinate = 0;    private int yCoordinate = 20;
    private Timer timer = new Timer(1000, new TimerListener());
    public MovingMessagePanel(String message) {
      this.message = message;
      // Start timer for animation
      timer.start();
      // Control animation speed using mouse buttons
      this.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseClicked(MouseEvent e) {
          int delay = timer.getDelay();
         if (e.getButton() == MouseEvent.BUTTON1)
           timer.setDelay(delay > 10 ? delay - 10 : 0);
         else if (e.getButton() == MouseEvent.BUTTON3)
           timer.setDelay(delay < 50000 ? delay + 10 : 50000);
       }     });    }
```

ตัวอย่าง : Animation

```
    @Override
    protected void paintComponent(Graphics g) {
      super.paintComponent(g);
      if (xCoordinate > getWidth()) {
        xCoordinate = -20;
      }
      xCoordinate += 5;
      g.drawString(message, xCoordinate, yCoordinate);
    }

    class TimerListener implements ActionListener {
      @Override
      public void actionPerformed(ActionEvent e) {
        repaint();
      }
    }
  }
}
```





ตัวอย่าง : ClockAnimation

```
import java.awt.event.*;
import javax.swing.*;
public class ClockAnimation extends JFrame {
  private StillClock clock = new StillClock();
  public ClockAnimation() {
    add(clock);
    // Create a timer with delay 1000 ms
    Timer timer = new Timer(1000, new TimerListener());
    timer.start();
  }

  private class TimerListener implements ActionListener {
    @Override /** Handle the action event */
    public void actionPerformed(ActionEvent e) {
      // Set new time and repaint the clock to display current time
      clock.setCurrentTime();
      clock.repaint();
    }
  }
```

ตัวอย่าง : ClockAnimation

```
/** Main method */
  public static void main(String[] args) {
    JFrame frame = new ClockAnimation();
    frame.setTitle("ClockAnimation");
    frame.setSize(200, 200);
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
  }
}
```